

Clase 5: Redes Neuronales

Diego Stalder
dstalder@ing.una.py

Facultad de Ingeniería
Universidad Nacional de Asunción

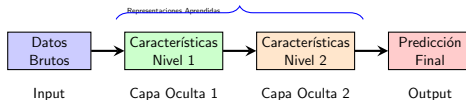
19 de Agosto de 2026

Inteligencia Artificial - 2026 Ingeniería Mecatrónica

El Papel de las Representaciones en ML

La Clave del Aprendizaje Automático

"Most machine learning methods work well because of human-designed representations and input features."



El Rol de las Redes Neuronales

- ML se convierte en **optimizar pesos** para hacer una predicción final.
- Las capas ocultas aprenden **representaciones automáticas**.
- Deep Learning: **aprendizaje de representaciones** jerárquicas.

Jerarquía de Representaciones

- **Bajo nivel:** Píxeles, bordes
- **Nivel medio:** Formas, texturas
- **Alto nivel:** Objetos, conceptos

Ejemplo: Visión por Computadora

- **Representación tradicional:** Píxeles → Características (SIFT, HOG)
- **Representación aprendida:** Píxeles → Bordes → Partes → Objetos

El Ciclo de Entrenamiento de una Red Neuronal

Flujo de Entrenamiento

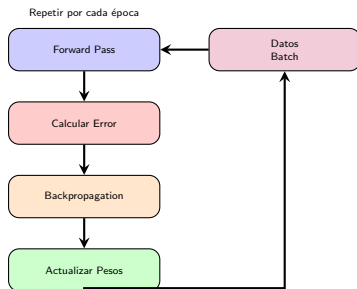
- 1 **Datos:** Muestras etiquetadas (batch)
- 2 **Forward Pass:** Propagar datos a través de la red
- 3 **Calcular Error:** Comparar predicciones con etiquetas
- 4 **Backpropagation:** Propagar el error hacia atrás
- 5 **Actualizar Pesos:** Usar el gradiente para mejorar

Ecuación de Actualización de Pesos

$$w_{\text{nuevo}} = w_{\text{viejo}} - \eta \cdot \frac{\partial \mathcal{L}}{\partial w_{\text{viejo}}}$$

donde:

- η : Learning Rate
- $\frac{\partial \mathcal{L}}{\partial w}$: Gradiente del error



Conceptos Clave

- **Epoch:** Una pasada completa por todos los datos
- **Batch:** Subconjunto de datos procesado
- **Iteración:** Una actualización de pesos

Forward Pass y Funciones de Activación

Propagación Hacia Adelante

1 Suma Ponderada:

$$z_j = \sum_{i=1}^n w_{ij} \cdot x_i + b_j$$

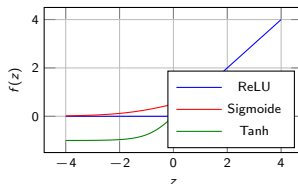
2 Activación:

$$a_j = f(z_j)$$

3 Predicción Final:

$$\hat{y} = f(W \cdot a + b)$$

Funciones de Activación



Funciones de Activación

Función	Fórmula	Rango
ReLU	$\max(0, x)$	$[0, \infty)$
Sigmoide	$\frac{1}{1+e^{-x}}$	$(0, 1)$
Tanh	$\frac{e^x - e^{-x}}{e^x + e^{-x}}$	$(-1, 1)$
Softmax	$\frac{e^{z_i}}{\sum_j e^{z_j}}$	$(0, 1)$

¿Cuál usar?

- ReLU: **Recomendada para capas ocultas**
- Sigmoide: Clasificación binaria
- Softmax: Clasificación multiclase
- Tanh: Datos centrados en cero

Ejemplo: Neurona

```
1 def neurona(x, w, b, activacion):  
2     z = np.dot(x, w) + b  
3     return activacion(z)  
4
```

Backpropagation: Propagación del Error

Concepto

- El error se propaga **hacia atrás** desde la salida
- Calcula la contribución de cada peso al error
- Usa la **regla de la cadena**

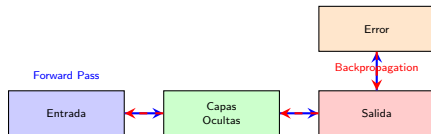
Regla de la Cadena

$$\frac{\partial \mathcal{L}}{\partial w_{ij}^{(l)}} = \frac{\partial \mathcal{L}}{\partial a_j^{(l)}} \cdot \frac{\partial a_j^{(l)}}{\partial z_j^{(l)}} \cdot \frac{\partial z_j^{(l)}}{\partial w_{ij}^{(l)}}$$

Gradiente del Error

$$\delta^{(l)} = (W^{(l+1)})^T \cdot \delta^{(l+1)} \odot f'(z^{(l)})$$

donde $\odot$ es el producto elemento a elemento.



Error Signal (Señal de Error)

- Capa de Salida:

$$\delta^{(L)} = \hat{y} - y$$

- Capas Ocultas:

$$\delta^{(l)} = (W^{(l+1)})^T \delta^{(l+1)} \odot f'(z^{(l)})$$

Descenso de Gradiente

Ideas Clave

- **Gradiente:** Dirección de máximo crecimiento
- **Descenso:** Moverse en dirección opuesta al gradiente
- **Learning Rate:** Tamaño del paso

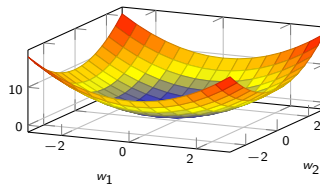
Ecuación de Actualización

$$w_{\text{nuevo}} = w_{\text{viejo}} - \eta \cdot \nabla \mathcal{L}(w_{\text{viejo}})$$

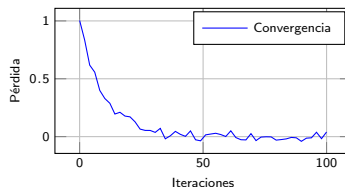
Variantes

- **Batch GD:** Usa todo el dataset
- **Stochastic GD:** Usa 1 muestra
- **Mini-Batch GD:** Usa un lote

Descenso de Gradiente



Convergencia



Ejemplo Sintético: Clasificación

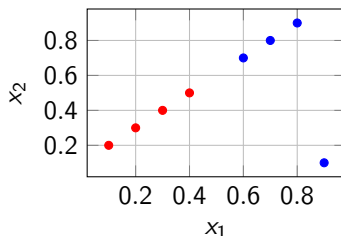
Problema: Two Moons

Clasificar puntos de dos clases en forma de lunas entrelazadas.

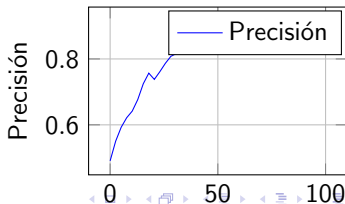
Código en Python

```
1 from sklearn.datasets import make_moons
2 from tensorflow.keras.models import Sequential
3 from tensorflow.keras.layers import Dense
4
5 # Generar datos
6 X, y = make_moons(n_samples=1000, noise=0.1)
7
8 # Crear modelo
9 model = Sequential([
10     Dense(128, activation='relu', input_shape=(2,)),
11     Dense(64, activation='relu'),
12     Dense(1, activation='sigmoid')
13 ])
14
15 model.compile(optimizer='adam',
16               loss='binary_crossentropy',
17               metrics=['accuracy'])
```

Datos: Two Moons



Evolución de la Precisión



Ejemplo Sintético: Regresión

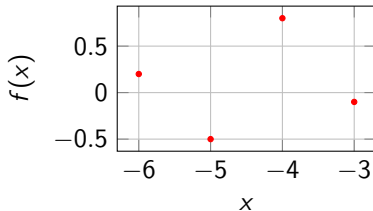
Problema: Aproximar
 $f(x) = \sin(x)$

Aprender la función subyacente a partir
de datos ruidosos.

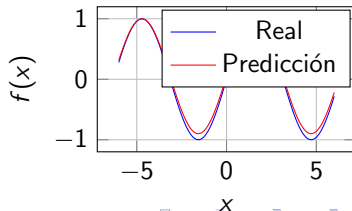
Código en Python

```
1 import numpy as np
2 from tensorflow.keras.models import Sequential
3 from tensorflow.keras.layers import Dense
4
5 # Generar datos
6 X = np.linspace(-2*np.pi, 2*np.pi, 1000)
7 y = np.sin(X) + 0.1*np.random.randn(1000)
8
9 # Crear modelo
10 model = Sequential([
11     Dense(64, activation='relu', input_shape=(1,)),
12     Dense(64, activation='relu'),
13     Dense(1) # Sin activacion para regresion
14 ])
15
16 model.compile(optimizer='adam', loss='mse')
```

Datos: $\sin(x) + \text{Ruido}$



Predicción del Modelo



¡Manos a la obra!

Objetivos de la Práctica

- 1 Implementar MLP para **clasificación** (Two Moons)
- 2 Implementar MLP para **regresión** (Seno)
- 3 Experimentar con diferentes **learning rates**
- 4 Experimentar con diferentes **batch sizes**
- 5 Observar el efecto en la convergencia y precisión

Tiempo Estimado: 30-45 minutos

Código base disponible en el notebook de la clase.

Preguntas Guía

- ¿Qué pasa si el learning rate es muy alto?
- ¿Qué pasa si el batch size es muy pequeño?
- ¿Cuál combinación de hiperparámetros funciona mejor?

Datos de Rendimiento Académico

Fuentes de Datos

- Archivos Excel de rendimiento académico
- Período: 2024 - 2025
- 3 archivos concatenados
- Formato anonimizado

Estructura del Dataset

- **64,295** registros totales
- **4,759** estudiantes únicos
- **326** asignaturas únicas
- **30** columnas
- 7 carreras de Ingeniería

Estructura

Mantiene la estructura del estudio CNMAC.

Columnas Principales

Columna	Descripción
ALUMNO.ID	Identificador del estudiante
Cod.Assign	Código de asignatura
Asignatura	Nombre de la asignatura
Cod.Car.Sec	Carrera-sección
Convocatoria	Número de convocatoria
Aprobado	S/N (Sí/No)
Primer.Par	Nota primer parcial
Segundo.Par	Nota segundo parcial
TPLab.	Nota trabajos/laboratorio
Lab.	Nota laboratorio
Proy.	Nota proyecto
Firma	Nota final

Estrategia de Modelado: "Pasa o No Pasa"

Objetivo

Predecir si un estudiante **pasa** o **no pasa** una asignatura.

Definición de Éxito

$$y = \begin{cases} 1 & \text{si Firma} \geq 5 \\ 0 & \text{si Firma} < 5 \end{cases}$$

Tipo de Problema

Clasificación Binaria

Características (Features)

- Primer.Par: Nota primer parcial
- Segundo.Par: Nota segundo parcial
- TPLab.: Trabajos prácticos
- Lab.: Laboratorio
- Proy.: Proyecto
- Convocatoria: Número de convocatoria
- Cod.Asign: Código de asignatura (categórica)
- Cod.Car.Sec: Carrera (categórica)

Esquema MLP

```
1 model = Sequential([
2     Dense(128, activation='relu', input_dim=n_features),
3     Dense(64, activation='relu'),
```

Métricas de Evaluación

Matriz de Confusión

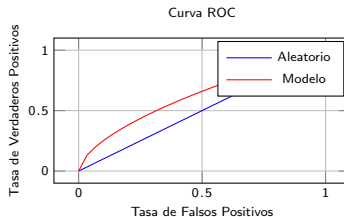
	Pasa	No Pasa
Pasa	VP	FN
No Pasa	FP	VN

Interpretación

- **Precisión:** ¿Cuántos de los que predije que pasan, realmente pasan?
- **Recall:** ¿Cuántos de los que realmente pasan, logré predecir?
- **F1-Score:** Media armónica de Precisión y Recall

Métricas

- **Precisión:** $\frac{VP}{VP+FP}$
- **Recall:** $\frac{VP}{VP+FN}$
- **F1-Score:** $2 \cdot \frac{\text{Precisión} \cdot \text{Recall}}{\text{Precisión} + \text{Recall}}$
- **Exactitud:** $\frac{VP+VN}{VP+VN+FP+FN}$



Lo que viene

- 1 Carga y exploración de los datos de rendimiento
- 2 Limpieza y preprocesamiento
 - Manejo de valores nulos
 - Codificación one-hot
 - Estandarización
- 3 Implementación del modelo "Pasa o No Pasa"
- 4 Experimentación con hiperparámetros
- 5 Evaluación y visualización de resultados

Entregable Final

Notebook Jupyter con todo el código, experimentos y conclusiones.

Conclusión

- Los MLP son una herramienta poderosa y flexible
- Los hiperparámetros son clave para el éxito del entrenamiento
- La aplicación a datos reales valida la utilidad práctica

¿Preguntas?

¿Qué conceptos necesitan más aclaración?

¿Cómo piensan aplicar esto a sus propios datos?

Material Adicional

- Artículo: "Understanding Deep Learning" - Goodfellow et al.
- Documentación: TensorFlow/Keras